South Valley National University

Course Book: Data Visualization

LEVEL4 Computer Science

Code: CS443

Content

Chapter 1 : introduction……..4

Chapter 2: Object-Oriented Programming …….9

Chapter 3: Hello flutter……18

Chapter 4:value widgets……41

Chapter 5 Navigation and Routing……52

Course Objectives

1. **Understand the fundamental concepts of mobile application development**, including architecture, design patterns, and lifecycle management for Android and iOS platforms.
2. **Gain practical knowledge of the Flutter framework**, and learn how to develop cross-platform mobile applications using the Dart programming language.
3. **Explore mobile UI design principles**, including responsive layouts, widgets hierarchy, navigation, and state management.
4. **Apply object-oriented programming (OOP) concepts** in mobile application development using Dart (classes, inheritance, polymorphism, encapsulation).
5. **Use Flutter's core components and widgets** to design and build interactive, visually appealing, and efficient user interfaces.
6. **Implement data handling and persistence techniques**, such as using local storage (SQLite, Shared Preferences) and integrating APIs for remote data access.
7. **Understand asynchronous programming and networking** in Flutter through the use of Futures, Streams, and HTTP requests.
8. **Develop skills in debugging, testing, and deploying** mobile applications to both Android and iOS platforms.
9. **Integrate third-party packages and plugins** to extend the functionality of mobile applications.
10. **Demonstrate teamwork, creativity, and problem-solving abilities** through the development of a full mobile application project using Flutter

Chapter 1

Introduction to the Dart Programming Language

1.1 Overview

Dart is a **modern, open-source, object-oriented programming language** developed by **Google** in 2011. It was designed to make application development faster and easier across multiple platforms, including **mobile, web, desktop, and backend systems**.

Dart is the **core language of Flutter**, Google's UI toolkit for building cross-platform applications from a single codebase.

Dart is a client-optimized language for developing fast apps that compile to native code

1.1.1 Why Google Created Dart

Google created Dart to address limitations found in JavaScript and to provide developers with:

- A more **structured and strongly typed** language.
- **Native performance** across platforms.
- A **unified language** that works for both frontend and backend development.
- Support for **JIT (Just-In-Time)** and **AOT (Ahead-Of-Time)** compilation

1.1.2 Why Dart for Flutter

Flutter depends on Dart for several reasons:

1. **Performance:** Dart compiles directly to native machine code.
2. **Hot Reload:** Dart's JIT mode allows developers to see changes instantly.
3. **Productivity:** Single codebase for Android, iOS, and Web.
4. **Safety:** Dart offers **Null Safety** to prevent common runtime errors.
5. **Object-Oriented:** Dart supports OOP principles used in modern app architecture

1.2 Features of Dart

Dart combines the power of compiled languages like C++ with the simplicity of dynamic languages like Python.

Main Features:

Feature	Description
Strong Typing	Enforces type safety and catches errors early.
Null Safety	Prevents null reference exceptions.
JIT and AOT Compilation	Enables fast development and optimized production builds.
Cross-Platform	Write once, run anywhere (mobile, web, desktop).
Object-Oriented	Everything in Dart is an object, even numbers and functions.
Garbage Collection	Automatic memory management.
Tooling	Includes Pub (package manager) and DevTools for debugging.

1.3 Basic Syntax

Every Dart program starts with the main() function.

- **Print Statement**
- **Description:**

  The print() function is used to display output in Dart.

Example: print("Hello, Dart!");

- **File Structure**

Description:

Dart files end with .dart and contain code such as functions and classes.

Example

```
// main.dart
void main() {
  print("Welcome to Dart");
}
```

- **Variables**

Description: A variable is used to store data. In Dart, you can declare variables using var, specific types, or dynamic.

Example:

```
var name = "Ali";
int age = 20;
double height = 1.75;
```

- **Data Types**

Description: Dart supports different data types:

- int for integers
- double for floating point numbers
- String for text
- bool for true/false values

```
int x = 5;
double y = 7.3;
String s = "Dart";
bool isOpen = true;
```

- **String Data Type**

Description: Represents text values.

Example:

```
String message = "Hello World";
print(message.length); // Output: 11
```

- **Int Data Type**

Description: Stores integer numbers (without decimal).

Example: int students = 30;

- **Double Data Type**

Description: Stores numbers with decimals.

Example: double price = 49.99;

- **Num Data Type**

Description: Can store either int or double.

Example: num a = 10;

a = 4.5;

- **Bool Data Type**

Description: Stores logical values: true or false.

Example: bool passed = false;

- **Dynamic**

Description: Stores any type and can change type at runtime.

Example

dynamic x = "Hi";

x = 42; // OK

- **var**

Description: Type inferred from the first assigned value, but cannot change type later.

Example:

var country = "Egypt";

// country = 5; // Error

- **Arithmetic Operators**

Description: Dart supports common arithmetic operations: addition (+), subtraction (-), multiplication (*), division (/), and modulo (%).

Example

int a = 8 + 2; // 10

int b = 7 - 3; // 4

int c = 5 * 2; // 10

double d = 10 / 3; // 3.333...

- **Dot Operator**

Description: The dot operator (.) is used to access object properties or methods.

Example:

String greeting = "Hello";

print(greeting.length); // 5

- **List**

Description: An ordered collection of items (array).

Example: List<int> numbers = [1, 2, 3];

- **Set**

Description: An unordered collection of unique items.

Example: Set<String> names = {'jac', 'Sam', 'lio'}; // {'Ali', 'Sara'}

- **Map**

Description: Collection of key-value pairs.

Example : Map<String, int> grades = {"jac": 90, "Sam": 95};

- **Operators (Comparison/Logical)**

Description: Compare values (==, !=, <, >) or combine logic (&&, ||).

Example

```
int a = 10, b = 20;
print(a < b && b > 15); // true
```

- **Conditional Statements**

Description: Control flow: if, else, switch

Example

```
if (a > 10) {
  print('Greater');
} else {
  print('Not greater');
}
```

- **Loops**

Description: Repeat code: for, while, do-while, for-in, for-each

Example:

```
for (int i = 0; i < 3; i++) {
  print(i);
}
```

- **Functions**

Description: Named blocks of code that do a task.

Example

```
int add(int a, int b) {
  return a + b;}
```

- **Parameters & Arguments**

Description: Values passed into functions.

Example:

```
void greet(String name) {
  print("Hi $name");
}
greet("Sam");
```

Optional & Named Parameters

Description: Parameters that can be omitted or specified by name.

Example

```
void show({String? name, int age = 18}) {
  print('$name - $age');
}
show(name: 'Sam');
```

Null Safety

Description:

Protects against null (missing) values by making nullability explicit.

Example

```
String? name; // can be null
name = null;
```

Chapter 2

Object-Oriented Programming (OOP) in Dart

2.1 Introduction to OOP

Object-Oriented Programming (OOP) is a programming paradigm that organizes software design around objects rather than functions or logic.

An object represents a real-world entity — it has attributes (data) and methods (behavior) that operate on that data.

Dart, being a fully object-oriented language, treats everything as an object, even primitive data types like numbers, strings, and booleans.

This makes OOP not just a feature but the core foundation of Dart and Flutter development

2.2 The Four Pillars of OOP

The four main principles of Object-Oriented Programming are:

- Encapsulation: Hiding internal data and exposing only necessary functionality.
- Abstraction: Simplifying complex systems by modeling only essential features.
- Inheritance: Allowing new classes to reuse properties and methods of existing ones.
- Polymorphism: Allowing objects to take many forms, usually through method overriding.

Dart also adds two additional advanced OOP concepts:

- **Mixins** (for modular code reuse)
- **Generics** (for type-safe flexibility)

2.3 Class and Object

A **class** is a **blueprint or template** that defines how an object should look and behave.

An **object** is an instance created from that blueprint.

Think of a class as a "design" and an object as the "real thing" built from it

Example:

```
class Human {
  int numberOfArms;
  double height;
  double weight;
  String hairColor;

  void read() {
    print("This human is reading.");
  }
}

void main() {
  Human jon = Human();
  jon.numberOfArms = 2;
  jon.height = 1.75;
  jon.weight = 70;
  jon.hairColor = "Black";

  jon.read();
}
```

- The **class Human** defines attributes (fields) and behaviors (methods).
- The **object jon** is an instance that stores its own data
-

Difference between Function and Method

Function	Method
Stand-alone piece of code, not tied to any class.	Belongs to a class or object.
Called directly.	Called using an object or class.
Example: `print("Hello");`	Example: `jon.read();`

2.3.1 Anonymous Objects

- In Dart (and in OOP generally), an **anonymous object** is an **object that is created without assigning it to a variable name**.

- It is often used when the object is needed **only once**, typically for **temporary operations** or **immediate method calls**.

Anonymous objects are extremely useful when:

- You don't need to reuse the object later.
- You just want to invoke a method quickly on a new instance

Simple Anonymous Object:

```
class Car {
  void start() {
    print("Car started");
  }
}

void main() {
  // Creating and using an anonymous object
  Car().start();
}
```

 Here, Car() creates an object **without a name**.

 The method .start() is called immediately on that unnamed instance.

 The object is automatically destroyed after the line executes, since no variable keeps a reference to it

2.4 Encapsulation

Encapsulation is the process of bundling data (attributes) and methods (functions) that operate on that data into a single unit (class), while controlling how that data is accessed or modified.

This ensures that data is protected from unintended modification — known as data hiding.

In Dart, encapsulation is achieved using:

Private members: Prefixing variable or method names with _ makes them private to their library.

Getters and Setters: Used to control access to private properties

Example of Encapsulation

```
class BankAccount {
  double _balance = 0.0; // Private variable

  void deposit(double amount) {
    if (amount > 0) {
      _balance += amount;
    }
  }

  void withdraw(double amount) {
    if (amount <= _balance) {
      _balance -= amount;
    } else {
      print("Insufficient funds!");
    }
  }

  double get balance => _balance; // Getter
}

void main() {
  var account = BankAccount();
  account.deposit(1000);
  account.withdraw(250);
  print("Current balance: ${account.balance}");
}
```

- The `_balance` variable is private, meaning it cannot be accessed directly outside the class.

  The user can only interact with it through methods (`deposit`, `withdraw`, and the `getter`)

2.5 Abstraction

Abstraction is the process of **hiding complex implementation details** and showing only the necessary features of an object. It simplifies code and makes it easier to maintain and extend.

In Dart, abstraction is achieved using **abstract classes** and **interfaces**.

Example :

```
abstract class Vehicle {
  void start(); // abstract method
  void stop();  // abstract method
}

class Car extends Vehicle {
  @override
  void start() {
    print("Car started");
  }

  @override
  void stop() {
    print("Car stopped");
  }
}

void main() {
  Vehicle myCar = Car();
  myCar.start();
  myCar.stop();
}
```

- Vehicle is an abstract class — it **cannot be instantiated directly**.
- The Car class **implements** the abstract methods with specific details

2.6 Inheritance

- Inheritance allows one class to **inherit** properties and methods from another.
- This supports **code reuse** and a **hierarchical relationship** between classes

Example of Inheritance:

```
class Animal {
  void eat() {
    print("Animal is eating");
  }
}

class Dog extends Animal {
  void bark() {
    print("Dog is barking");
  }
}

void main() {
  Dog dog = Dog();
  dog.eat(); // Inherited method
  dog.bark(); // Child method
}
```

- Dog inherits eat() from Animal.
- This avoids rewriting the same behavior multiple times

2.7 Polymorphism

- Polymorphism (meaning “many forms”) allows objects of different types to be treated as objects of a common superclass.
- It enables **method overriding** — redefining inherited methods to perform specific actions.

Example of polymorphism:

```
class Animal {
  void makeSound() {
    print("Animal makes a sound");
  }
}

class Cat extends Animal {
  @override
  void makeSound() {
    print("Meow");
  }
}

class Dog extends Animal {
  @override
  void makeSound() {
    print("Woof");
  }
}

void main() {
  List<Animal> animals = [Cat(), Dog()];
  for (var a in animals) {
    a.makeSound(); // Calls the appropriate overridden method
  }
}
```

2.8 Mixins

- A **mixin** is a way to **reuse code across multiple classes** without using inheritance.
- A mixin is like a partial class that can be “mixed in” to another class

```
mixin Swimmer {
  void swim() => print("Swimming...");
}

mixin Flyer {
  void fly() => print("Flying...");
}

class Duck with Swimmer, Flyer {
  void showAbilities() {
    swim();
    fly();
  }
}

void main() {
  Duck d = Duck();
  d.showAbilities();
}
```

- The Duck class combines two behaviors (swim + fly) using mixins.
- This avoids complex inheritance chains.

2.9 Enumeration (Enums):

- An **enum** defines a **set of named constant values**.
- It's useful when a variable can only take one of a few predefined options.

```
enum Day { Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday }

void main() {
  Day today = Day.Friday;

  switch (today) {
    case Day.Friday:
      print("Weekend is near!");
      break;
    default:
      print("Working day.");
  }
}
```

2.10 Generics

- Generics allow you to create **classes and methods** that work with **different data types** while maintaining **type safety**.
- They make code reusable and prevent runtime type errors

Example of Generics:

```
class Box<T> {
  T content;

  Box(this.content);

  void showContent() {
    print("Content: $content");
  }
}

void main() {
  var intBox = Box<int>(100);
  var strBox = Box<String>("Hello Dart");

  intBox.showContent();
  strBox.showContent();
}
```

Chapter 3

Hello Flutter

3.1 What is Flutter?

Flutter is a set of tooling that allows us to create beautiful apps that run on iOS, Android, the Web, and desktop.

Flutter is:

- Free (as in free beer. No cost)
- Open source (that's the other sense of the word "free")
- Backed by and originated at Google
- Being enhanced and maintained by a team of developers at Google and hundreds of non-Google contributors around the globe
- Currently being used by thousands of developers in organizations across the world for production apps
- Fast because it compiles to truly native apps that don't use crutches like WebViews and JavaScript bridges.
- Written one place and compiled to a web app for billions of browsers, an iOS app for iPhones and iPads, and an Android app for all of the rest of the phones and tablets out there

3.2 Why Flutter?

Google's mission with Flutter is ...

To build a better way to develop for mobile

Notice what is <u>not</u> in that mission. There's no mention of Android (which is also owned by Google) nor of iOS nor of the Web. Flutter's goal is to create a better way to develop for all devices. In other words, Flutter should be better to create iOS apps than Swift. It should be better to create Android apps than Kotlin. It should be better to create web apps than HTML/JavaScript. And if you get all of those things simultaneously with one codebase, all the better.

The Flutter team has succeeded spectacularly with this mission.
As proof, Eric Seidel offers this example. The Google CRM team used Flutter to build an internal Android app and did it **three times** faster than with their traditional Android toolchain!

But it turns out that Flutter isn't the only game in town for cross-platform. You have other options.

Everything Is Widgets

Let's pretend that you are an insanely talented Lego nerd and got offered one of the few coveted jobs as a Lego Master Builder. Congrats! Let's also say that your first assignment is to build a six-foot-tall Thor made from 26,000 Legos (Figure 3-1).

Figure 3-1. A Lego Thor. The author snapped this picture at a movie theater once

How would you go about doing that? Ponder that for a minute. Go ahead, we'll wait.

Would you just start grabbing bricks and putting them together?
Probably not. Would you lay out the soles of Thor's feet and build from the bottom up? Again, no. Here's my guess as to your common-sense strategy:

1. You'd get a vision of what you're building. Figure the whole thing out.
2. Realize that the entire project is too complex to build at once.
3. Break the project into sections (legs, left arm, right arm, torso, left sword, right sword, helmet, cape, head).
4. Realize that each of them is still too complex.
5. For each section, you break it into sub-sections.
6. Repeat steps 4 and 5 until you've got simple enough components that each is easy to understand, build, and maintain – for you and for any teammates that you may have.
7. Create each simple component.
8. Combine simple components to form the larger, more complex components.
9. Repeat steps 7 and 8 until you've got your entire project created.

This process has a name: *componentization*, and is exactly the thought process we'll go through with our Flutter projects.

3.3 Built-in Flutter widgets

Flutter's foundational widgets are the building blocks of everything we create and there are tons of them – about 160 at last count. This is a lot of widgets for you and I to keep track. But if you mentally organize them, it becomes much more manageable.

They fall into these major categories:

- Value widgets
- Layout widgets
- Navigation widgets
- Other widgets

3.3.1 Value widgets

Certain widgets hold a value, maybe values that came from local storage, a service on the Internet, or from the user themselves. These are used to display values to the user and to get values from the user into the app.
The seminal example is the Text widget which displays a little bit of text. Another is the Image widget which displays a .jpg, .png, or another picture. Here are some more value widgets:

Checkbox	FormField	RefreshIndicator
CircularProgressIndicator	Icon	RichText
Date & Time Pickers	IconButton	Slider
DataTable	Image	Switch
DropdownButton	LinearProgressIndicator	Text
FlatButton	PopupMenuButton	TextField
FloatingActionButton	Radio	Tooltip
FlutterLogo	RaisedButton	
Form	RawImage	

We'll explore value widgets in more detail in the next chapter.

3.3.2 Layout widgets

Layout widgets give us tons of control in making our scene lay out properly – placing widgets side by side or above and beneath, making them scrollable, making them wrap, determining the space around widgets so they don't feel crowded, and so on:

Align	FittedBox	Padding
AppBar	Flow	PageView
AspectRatio	FractionallySizedBox	Placeholder
Baseline	GridView	Row
BottomSheet	IndexedStack	Scaffold
ButtonBar	IntrinsicHeight	Scrollable
Card	IntrinsicWidth	Scrollbar
Center	LayoutBuilder	SingleChildScrollView
Column	LimitedBox	SizedBox
ConstrainedBox	ListBody	SizedOverflowBox
Container	ListTile	SliverAppBar
CustomMultiChildLayout	ListView	SnackBar
Divider	MediaQuery	Stack
Expanded	NestedScrollview	Table
ExpansionPanel	OverflowBox	Wrap

3.3.3 Navigation widgets

When your app has multiple scenes ("screens," "pages," whatever you want to call them), you'll need some way to move between them. That's where Navigation widgets come in. These will control how your user sees one scene and then moves to the next. Usually this is done when the user taps a button.

And sometimes the navigation button is located on a tab bar or in a drawer that slides in from the left side of the screen. Here are some navigation widgets:

AlertDialog	MaterialApp	TabBar
BottomNavigationBar	Navigator	TabBarView
Drawer	SimpleDialog	

How to create your own stateless widgets?

So we know that we will be composing these built-in widgets to form our own custom widgets which will then be composed with other built-in widgets to eventually form an app.

Widgets are masterfully designed because each widget is easy to

understand and therefore easy to maintain. Widgets are abstract from the outside while being logical and predictable on the inside. They are a dream to work with.

Every widget is a class that can have properties and methods. Every widget can have a constructor with zero or more parameters. And most importantly, every widget has a build method which receives a BuildContext and returns a single Flutter widget. If you're ever wondering how a widget got to look the way it does, locate its build method:

```
class RootWidget extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Text('Hello world');
  }
}
```

In this hello world example which we repeated from earlier in the chapter, we're displaying a Text widget (Figure 3-2). A single inner widget works but real-world apps will be a whole lot more complex. The root widget could be composed of many other subwidgets:

```
class FancyHelloWidget extends StatelessWidget {
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(
          title: Text("A fancier app"),
        ),
        body: Container(
          alignment: Alignment.center,
          child: Text("Hello world"),
        ),
        floatingActionButton: FloatingActionButton(
          child: Icon(Icons.thumb_up),
          onPressed: () => {},
        ),
      ),
    );
  }
}
```

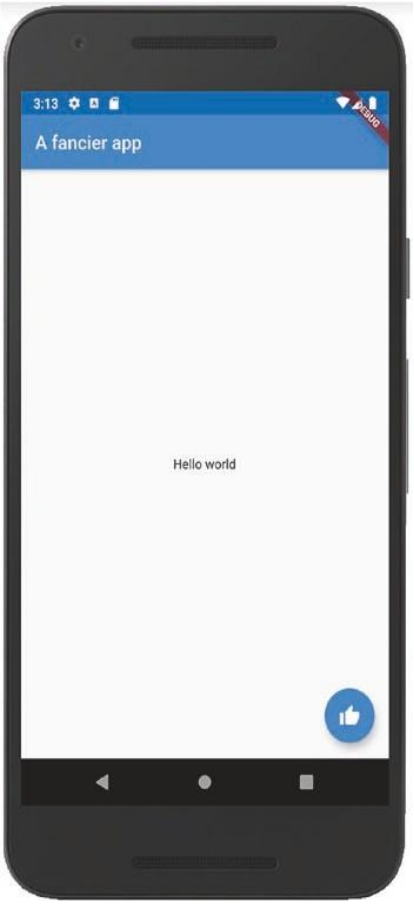

Figure 3-2. *The app created by this simple widget*

So as you can see, the build method is returning a single widget, a MaterialApp, but it contains a Scaffold which contains three subwidgets: an AppBar, a Container, and a FloatingActionButton (Figure 3-3). Each of those in turn contains sub-subwidgets of their own.

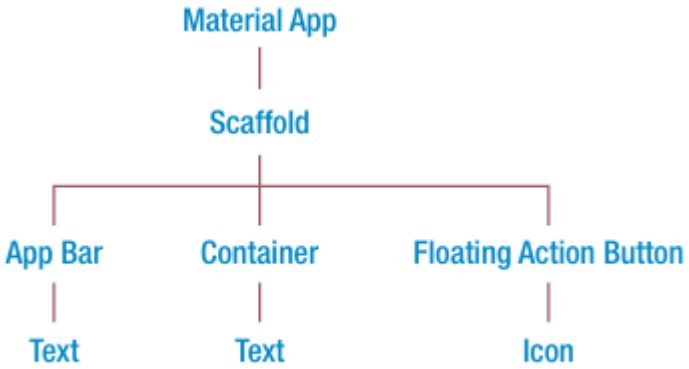

Figure 3-3. *The widget tree from our example app above*

This is how your build method will always work. It will return a single, massive, nested expression. It is widgets inside widgets inside widgets that

enable you to create your own elaborate custom widget.

3.4 Stateless and Stateful Widgets

In Flutter, **everything is a widget**.

Every visual element, layout, and even the app itself is represented as a widget.

However, how widgets handle **data changes** defines their type:

- Some widgets never change — these are **Stateless Widgets**.
- Others can change their internal data over time — these are **Stateful Widgets**

➢ **Stateless Widgets**

A **Stateless Widget** is immutable. Once created, its data and appearance remain constant throughout its lifetime.

- If it needs to display new data, it must be **rebuilt** by its parent widget with new parameters.

Example: The `Person` Widget

```
class Person extends StatelessWidget {
  final String firstName;
  final String lastName;

  Person({this.firstName, this.lastName});

  @override
  Widget build(BuildContext context) {
    return Container(child: Text('$firstName $lastName'));
  }
}
```

In this example, the `Person` widget simply displays the given name and surname. It does **not** store or update any data internally — therefore it is **stateless**

Characteristics

- Depends only on external input data.
- Does not maintain internal state.
- Rebuilt only when the parent provides new data.
- Lightweight and easy to test.

➢ **Stateful Widgets**

- A **StatefulWidget** is a widget that maintains **mutable state** — data that can change during the widget's lifetime.
- When that data changes, the widget calls **`setState()`**, which triggers a rebuild of its visual representation.

Choosing Between Stateless and Stateful Widgets

When deciding which type of widget to use, a simple guideline can help:

Start your design using a StatelessWidget whenever possible.
If the widget needs to change its data during runtime — for example, when responding to user actions — then it should be converted into a StatefulWidget.
In general, the fewer stateful widgets you have, the better your application's structure will be.
Reducing state makes the code simpler, more predictable, and easier to maintain and test.
Therefore, managing state only when necessary is considered a best practice in Flutter development

- **Advanced Note: Inherited Widget**

There is also a third, more advanced type: the **InheritedWidget**.
It allows you to **share data down the widget tree** so that descendant widgets can access it without passing it manually through constructors. This concept is used in many Flutter features such as **Theme**, **Media Query**, and **Provider.**

- **Understanding "State"**

State refers to data within a widget that can change during its lifetime and whose change requires a **re-render.**
Flutter provides many built-in widgets that are already **stateful**, such as:

- TextField, Form, Switch, Slider
- Scaffold, Navigator, Image
- Checkbox, Radio, DropdownButton, and many more

- **Key Takeaways**

- Use StatelessWidget for static, unchanging UI components.

- Use StatefulWidget when your widget must react to user input or internal events.
- Prefer immutability and minimize local state — this leads to cleaner, more predictable Flutter code.
- Remember that state management becomes crucial in large apps — Flutter provides many tools for this, such as InheritedWidget, Provider, and Riverpod.

Chapter 4

Value Widgets

4.1 Introduction

In Flutter, everything is a widget. Widgets are the fundamental building blocks of every user interface you design. Among the different kinds of widgets available, **Value Widgets** play a unique and foundational role. They are the widgets responsible for *displaying information* and *capturing user input*.

Unlike layout widgets (which control structure and positioning) or navigation widgets (which manage screen transitions), **Value Widgets** deal directly with *data representation*. They either show a value (such as text or an image) or collect a value (like a form field or checkbox).

In other words, Value Widgets serve as the **visual connection between the application logic (data/state) and the user interface**. Whenever you see something on the screen that displays a piece of data — a line of text, a photo, an icon — you are using a value widget.

Examples include:

- Text — displays a string value.
- Icon — represents symbolic data visually.
- Image — shows bitmap or network-based graphical data.
- TextField, Checkbox, Radio, Switch — accept and represent user input values.
- Form — groups and manages input widgets collectively.

Note: The naming "Value Widget" reflects that these widgets *depend on or produce a value*. This concept aligns with the reactive programming model of Flutter, where widgets rebuild when their underlying values change.

4.2 The Text Widget

The Text widget is one of the most commonly used value widgets. It displays a single line or multiple lines of text, optionally styled, aligned, or wrapped according to layout constraints.

Example:

```
Text(
  'Welcome to Flutter!',
  style: TextStyle(
    fontSize: 22,
    fontWeight: FontWeight.bold,
    color: Colors.blueAccent,
  ),
  textAlign: TextAlign.center,
)
```

This widget renders the string *"Welcome to Flutter!"* in bold blue text, centered horizontally.

TextStyle Parameters

The TextStyle class allows customization of text appearance:

- fontSize – adjusts size in logical pixels.
- fontWeight – makes text bold, light, etc.
- fontStyle – italic or normal.
- color – defines the color.
- letterSpacing, wordSpacing – add spacing adjustments.
- decoration – underline, overline, or strike-through.

RichText and TextSpan

For multiple styles in a single paragraph, Flutter uses RichText and TextSpan

```
RichText(
  text: TextSpan(
    text: 'Hello ',
    style: TextStyle(color: Colors.black, fontSize: 20),
    children: [
      TextSpan(
        text: 'Flutter',
        style: TextStyle(color: Colors.blue, fontWeight: FontWeight.bold),
      ),
    ],
  ),
)
```

Note: Insert a screenshot showing both a simple Text and a styled RichText widget, with arrows labeling TextSpan regions.

4.3 The Icon Widget

Icons communicate ideas visually. Flutter's `Icon` widget displays graphical symbols from predefined sets such as `Icons` (Material Design) or `CupertinoIcons` (for iOS). Flutter comes with a rich set
of built-in icons (Figure 4-1)

Figure 4-1. An assortment of Flutter's built-in widgets in random colors

Example:

```
Icon(
  Icons.favorite,
  color: Colors.redAccent,
  size: 40,
)
```

This renders a red heart symbol using the Material icon set.

Custom Icons

You can use custom icons by including .ttf font files and defining them in your project's pubspec.yaml.

Note: Icons reduce cognitive load for users and improve UI intuitiveness. In Flutter, icons are vector-based, ensuring scalability without pixelation.

- Add an image showing an app bar with three icons: home, favorite, and settings

4.4 The Image Widget

Images enrich the visual experience of apps. Flutter supports several image sources:

- **Asset images** (bundled with the app)
- **Network images** (downloaded from the internet)
- **File images** (from local storage)
- **Memory images** (from raw byte data)

Examples:

```
// From local assets
Image.asset('assets/images/logo.png');

// From a network URL
Image.network('https://flutter.dev/images/flutter-logo-sharing.png');

// From device file system
Image.file(File('/storage/emulated/0/Pictures/sample.jpg'));
```

4.4.1 Sizing an image

Images are nearly always put in a container. Not that this is a requirement, it's just that I can't imagine a real-world use case where it won't be inside another widget. The container has a say in the size that an image is drawn. It would be an amazing coincidence if the Image's natural size fit its container's size perfectly. Instead, Flutter's layout engine will shrink the image to fit its container, but not grow it. This fit is called *BoxFit. scaleDown*, and it makes sense for the default behavior. But what other options are available and how do we decide which to use? Table 4-1 provides your BoxFit options.

Table 4-1. BoxFit options

fill	Stretch it so that both the width and the height fit exactly. Distort the image	
cover	Shrink or grow until the space is filled. The top/bottom or sides will be clipped	
fitHeight	Make the height fit exactly. Clip the width or add extra space as needed	
fitWidth	Make the width fit. Clip the height or add extra space as needed	
contain	Shrink until both the height and the width fit. There will be extra space on the top/ bottom or sides	

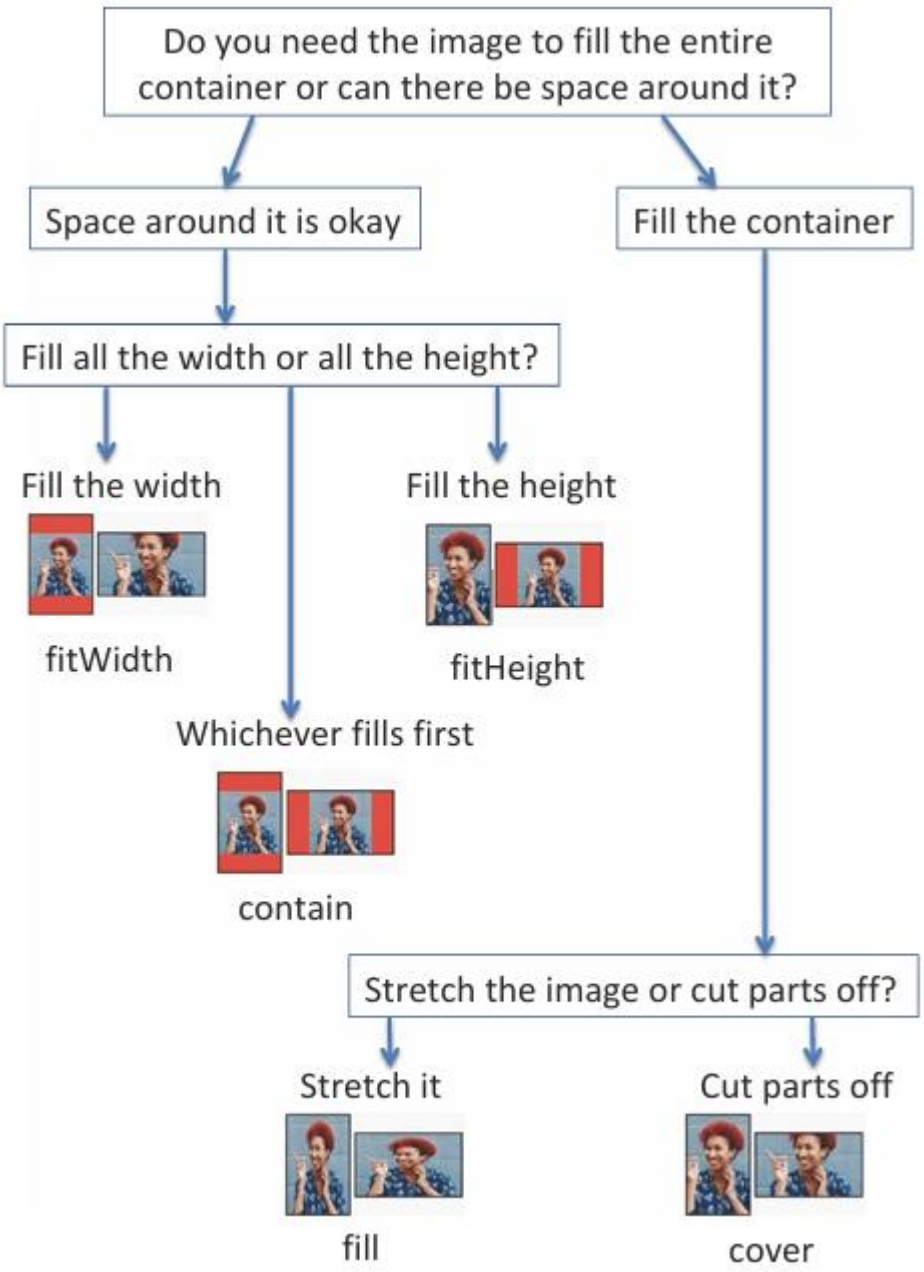

Figure 4-3. *How to decide an image's fit*

To specify the fit, you'll set the fit property.

Image.asset('assets/images/woman.jpg',

fit: BoxFit.contain,),

4.5 Input Widgets

In web development, we use elements like <form>, <input>, and <select> to collect user data. Flutter provides similar widgets for data entry, but they work differently and require more setup. The advantage is that they are safer and give you greater control.

Input widgets in Flutter **don't maintain their own state** — you must handle the state manually. They also **don't interact with each other** unless they are grouped inside a **Form widget**, which helps manage and validate multiple inputs together.

 Note:

Input widgets change their state as users interact with them. Therefore, they should always be used inside a **Stateful Widget**.

Text Fields

The **TextField** widget is used when you need a single text box for user input.

You can detect changes while typing using the **on Changed** property, which gives you the text as a String. To set or read an initial value, use a **Text Editing Controller**.

Example:

```
var controller = TextEditingController(text: "Initial value");
TextField(controller: controller);
```

You can customize the appearance of a TextField with the **Input Decoration** widget, which allows:

- **labelText** – title above the field
- **hintText** – light placeholder text inside
- **errorText** – error message below
- **prefix/suffix text or icons** – extra content around the text box

To hide characters for passwords, set obscureText: true.

You can also choose the **keyboard type**, such as numbers or email, using keyboardType:.

Finally, use **input Formatters** to restrict or validate input, e.g. allow only digits or limit text length.

Example:

```
TextField(
  inputFormatters: [
    FilteringTextInputFormatter.allow(RegExp('[0-9 -]')),
    LengthLimitingTextInputFormatter(16),
  ],
  decoration: InputDecoration(labelText: 'Credit Card'),
);
```

Flutter's layout algorithm

In your custom widget, you have a root widget at the top of your main method. It has branches and branches of branches and on and on. Let's call this the widget tree (Figure4-4). Flutter has to decide how big to make each widget in the tree. It does so by asking each widget how big it would prefer to be and asking its parent if that is okay.

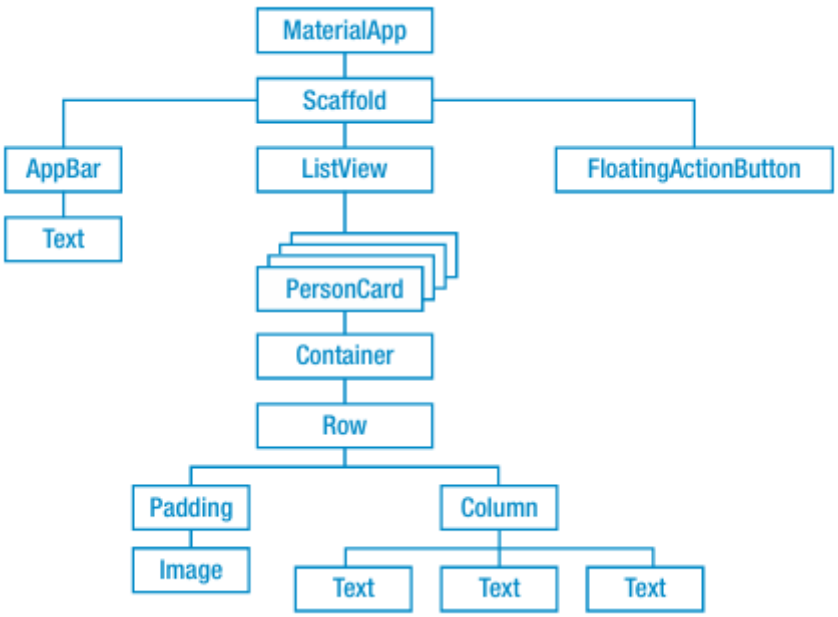

Figure 4-4. Every scene has a widget tree

Flutter travels down the tree starting at the root. It reads the constraint of the root widget. "What is the tallest you can be? And the widest?" It remembers them and then looks for any children. For each child, it communicates its BoxConstraints to

them and then travels to the grandchildren. It keeps doing this all the way to the end of every branch. We call this the leaf level.

It then asks each leaf how big it would prefer to be. “What is your favorite height? What is your favorite width?” It allows the leaf to be drawn at its preferred size within the constraints of all of its ancestors. If the preferred size is too big, Flutter clips it at runtime – something we really try to avoid! If the preferred size is too small, Flutter pads it with extra space until it fits.

It then goes back up a level and tries to fit those branches inside their common parent which has its own constraints. And so on all the way back up to the top.

The result is that each child gets to be its favorite height and width – as long as its parent allows it. And no parent has a final size until all of its child do.

Tip

Another situation you’re going to come across is when you have a widget whose renderBox is larger than its BoxConstraints. in other words, this single widget can’t fit inside its parent. the solution for that problem is occasionally a FittedBox,3 a widget that shrinks it’s child to fit. By default, you’ll get a centered widget that is scaled down until it just fits both horizontally and vertically, but you have the options to align it vertically/horizontally and to stretch it or clip top/ bottom or left/right.

So you can see how we’d get the “unbounded height” error. If we had a child who tries to be as large as it can and it doesn’t have a parent to tell it to stop, Flutter panics because it is now infinitely tall. To solve the problem, that child simply needs a parent to tell it to stop growing. A LimiteBox() widget’s main characteristic is to do exactly that; it tells a child just how big it is allowed to get if the parent refuses to. And Flutter has a ton of widgets to control size and position. For the rest of this chapter, we’re going to study the most critical of those layout widgets – the ones you absolutely must know. We’ll start with Row and Colum.

4.6 Layout Widgets

In Flutter, **Layout Widgets** are used to arrange and position other widgets on the screen. While Value Widgets display data, Layout Widgets control **how and where** those widgets appear. The most common layout widgets are **Row**, **Column**, **Container**, and **Stack**. Together, they form the foundation of every Flutter interface

Putting widgets next to or below others

Row and Column, as the names suggest, are made for laying out widgets side by side (Row, Figure 4-5) or above and below (Column, Figure 4-6). Other than how they lay out their children, they're nearly identical.

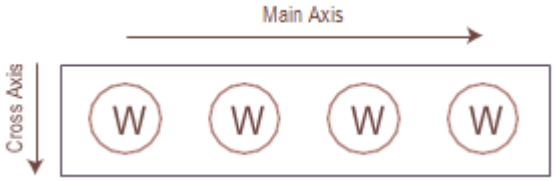

Figure 4-5. A Row widget lays out side by side

```
Row(
  children: <Widget>[
    Widget(),
    Widget(),
    Widget(),
    Widget(),
  ],
),
```

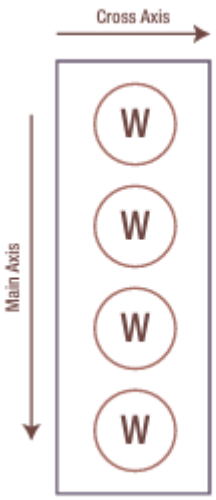

Figure 4-6. A Column widget lays out above and below

```
Column(
  children: <Widget>[
    Widget(),
    Widget(),

    Widget(),
    Widget(),
  ],
),
```

Notice that they both have a children property which is an array of Widgets. All widgets in the children array will be displayed in the order you add them. You can even have rows inside columns and vice versa as many levels deep as you like. In this way you can create nearly any layout imaginable in any app. Rows and columns will be your go-to layout widgets. Yes, there are others, but these two are your first calls.

Note

Occasionally you'll want two things above and below when the device is in landscape and side by side when in portrait. so you want them in a row at times and a column at others. this is when you'll use a Flex widget which can do both. it has an orientation property that will be conditional:

```
Flex(
  direction:
    MediaQuery.of(context).orientation ==
    Orientation.landscape ?
       Axis.horizontal : Axis.vertical,
  children: <Widget>[
    SomeWidget(),
    SomeWidget(),
    SomeWidget(),
  ],
),
```

This doesn't happen as often as you might think. Use it sparingly.

Your widgets will never fit!

It would be an overwhelming coincidence if the elements fit perfectly in a scene. And if they ever fit perfectly, as soon as the app is run on a different screen size or rotated from portrait to landscape, that will change. So we need to handle two situations:

1. What if there's extra space left over? (more screen than pixels taken up by the widgets)
2. What if there's not enough space? (too many widgets in a given space)

These are both likely to happen simultaneously on different parts of your scene. Let's tackle leftover space first.

What if there's extra space left over?

This is an easy problem to solve. The only question you really need to answer is how to distribute the extra room. How much space do you want to allocate around each of the other widgets? You have several options. The easiest and quickest is to use mainAxisAlignment and crossAxisAlignment.

MainAxisAlignment

MainAxisAlignment is a property of the Row or Column (Figure 6-14). With it you control how the extra space is allocated with respect to the widgets along the main axis – vertical for columns and horizontal for rows:

```
child: Column(
  mainAxisAlignment: MainAxisAlignment.spaceEvenly,
  children: <Widget>[
    SubWidget(),
    SubWidget(),
```

You have a few choice :

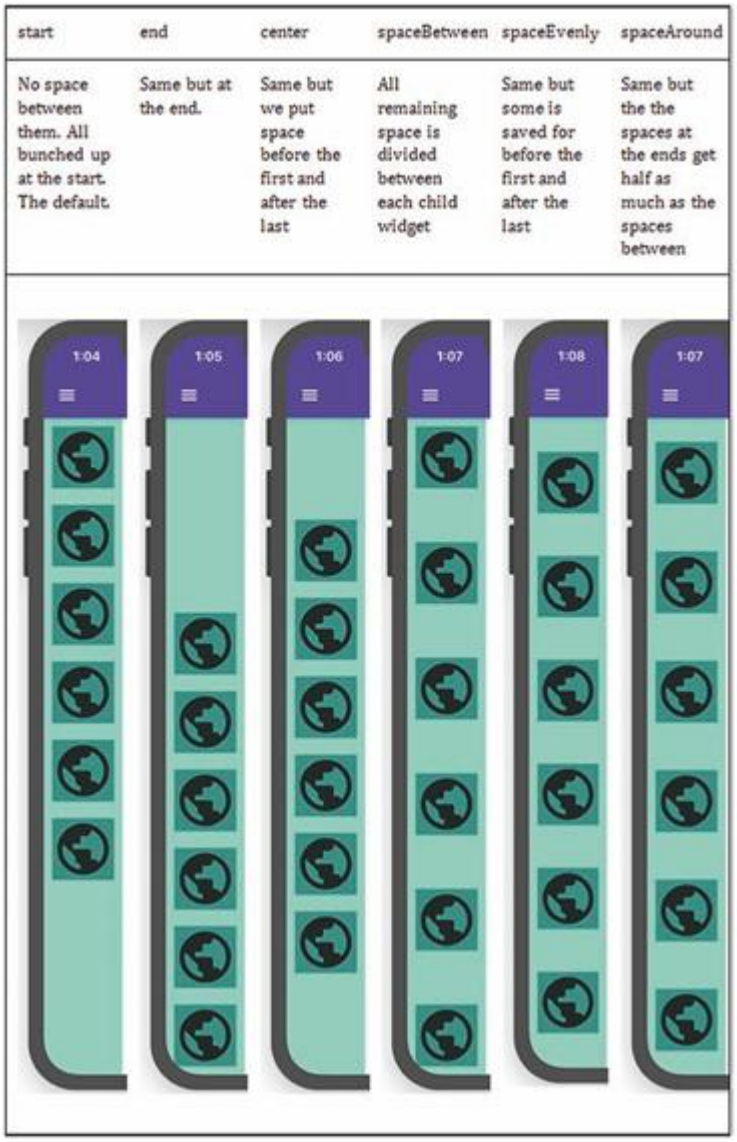

Figure 4-7. mainAxisAlignment says how to distribute the extra space along the main axis

crossAxisAlignment

crossAxisAlignment is also a property of the Row or Column; it decides where to put the extra space if the widgets are of different heights in a row or widths in a column (Figure 4-8). Your options are

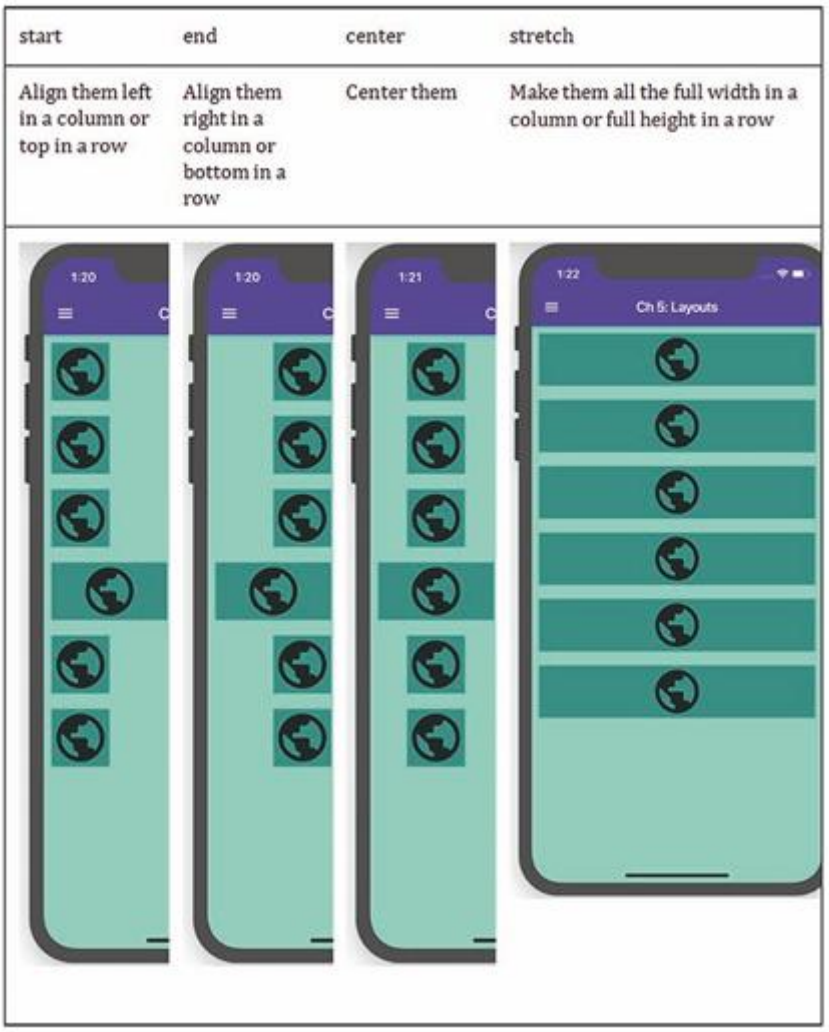

Figure 4-8. crossAxisAlignment says how to distribute extra space along the cross axis

Expanded widget

mainAxisAlignment is awesome if the spacing is cut and dried – you want equal spacing somehow. But what if you don't want spacing at all? What if you want the widgets to expand to fill the remaining space? Expanded widget to the rescue (Figure 4-9). Let's take this code for an example:

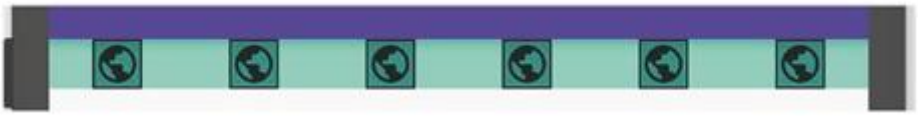

Figure 4-9. This Row widget has lots of extra space

```
Row(
  mainAxisAlignment: MainAxisAlignment.spaceAround,
  children: <Widget>[
    SubWidget(),
    SubWidget(),
    SubWidget(),
    SubWidget(),
    SubWidget(),
    SubWidget(),
  ],
```

When you wrap a Row/Column's child in an Expanded widget (Figure 4-10), it makes that child flexible, meaning that if there is extra space, it will stretch along the main axis to fill that space. Here's the same thing but with an Expanded() around the second widget:

```
Row(
  mainAxisAlignment: MainAxisAlignment.spaceAround,
  children: <Widget>[
    SubWidget(),
    Expanded(child: SubWidget()),
    SubWidget(),
    SubWidget(),
    SubWidget(),
    SubWidget(),
  ],
```

Figure 4-10. The second widget is wrapped in an Expanded

Note that the mainAxisAlignment now makes no difference because there is no extra space. It's all eaten up by the Expanded. What if we add another Expanded? Let's put one around the third and fourth widgets also (Figure 4-11):

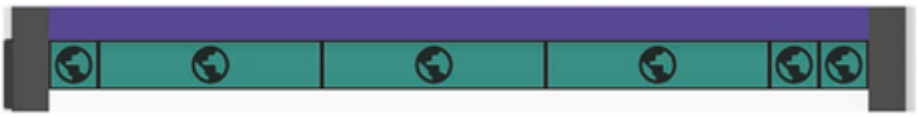

Figure 4-11. Expandeds will divide the free space among them

```
Row(
  children: <Widget>[
    SubWidget(),
    Expanded(child: SubWidget()),
    Expanded(child: SubWidget()),
    Expanded(child: SubWidget()),

    SubWidget(),
    SubWidget(),
  ],
```

What if there's not enough space?

We've tackled the situations where there is too much space and how to control where that extra space is allocated. But what if there is too little space? Like we are trying to squeeze too many widgets into too small a row or column? Unless you do something about it, Flutter will clip the widgets which looks terrible and worse, may hide some widgets from the user. So what do you do? You allow scrolling! While it is possible to scroll in both directions, it creates some serious usability issues. So we recommend that you stick to scrolling in one direction only and that it usually be vertical scrolling. The easiest way to scroll is with a ListView.

The ListView widget

ListView has actually has four different ways to use it:

1. new ListView – Normal use. It has a children property that takes a collection of static widgets.

2. ListView.builder – For dynamically creating children from a list of items.

3. ListView.separated – Like builder but also puts a widget ∗between∗ each item. Great for inserting ads in the list periodically. Read more at http://bit. ly/flutter_listview_separated.

4. ListView.custom – For rolling your own advanced listviews. Read more at http://bit.ly/flutter_ listview_custom.

Container widget and the box model

Flutter has borrowed heavily from other technologies including HTML and the Web which have the ideas of borders, padding, and margin. These are collectively called

the box model. They're used to create pleasant-to-the- eyes spacing around and between screen elements. It's a battle-proven concept that has worked great for the Web so why not borrow it for Flutter?

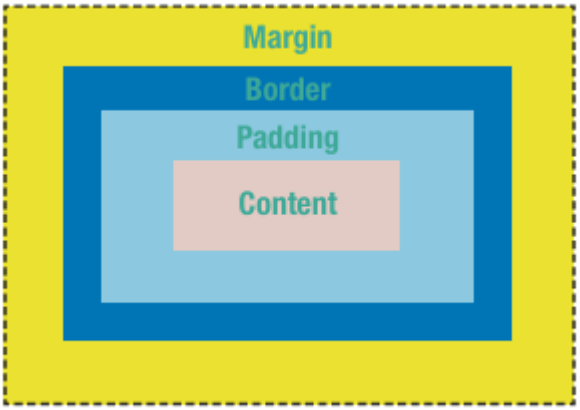

Figure 4-12. The box model defines padding, border, and margin

Let's say that we have a sized image that we want framed so to speak with a padding of 8, a margin of 10, and a border of 1. Flutter newcomers might try this first:

```
Image.network(
  _peopleList[i]['picture']['thumbnail'],
  padding: 8.0,
  margin: 10.0,
  border: 1.0,
),
```

This would not work since Image widgets don't have a padding, margin, or borders. But you know what does? Containers!

Web developers often apply these things by wrapping elements in a generic container called a < div> and then applying styles to create pleasant spacing for our web pages.

Flutter doesn't have a < div>, but it does have a div-like widget called a Container which ... well ... contains other things. In fact, its entire life purpose is to apply layout and styles to the things inside of it. An HTML can hold multiple things, but a Flutter Container only holds one child. It has properties called padding, margin, and decoration. We'll leave decoration for the styles chapter, but padding and margin are awfully handy for creating nice-looking spacing:

```
Container(
  padding: EdgeInsets.all(8.0),
  margin: EdgeInsets.all(10.0),
  decoration: BoxDecoration(border: Border.all(width: 1.0)),
  child:   Image.network(thePicture),
  // Container has *lots* of other properties, many of which
  // we'll cover in the Styles chapter.
),
```

Tip

Margin and padding might have been easier to learn if they had just allowed us to list four number values representing the four sides. (they couldn't make it easy, could they?) instead, we use a helper widget called edgeinsets.

• edgeinsets.all(8.0) – same value applied to all four sides evenly.

• edgeinsets.symmetric(horizontal: 7.0, vertical: 5.0) – top and bottom are the same. Left and right are the same.

• edgeinsets.only(top: 20.0, bottom: 40.0, left: 10.0, right: 30.0) – Left, top, right bottom can all be different.

• edgeinsets.fromLtrB(10.0, 20.0, 30.0, 40.0) – same as the preceding one but less typing. also note that if you want padding only – no other formatting – the padding widget is a shorthand.

```
Container(
 padding: EdgeInsets.all(5),
 child: Text("foo"),
),
```

```
Padding(
padding: EdgeInsets.all(5),
child: Text("foo"),
),
```

These two are equivalent.

Special layout widgets

Like we said at the top of the chapter, we've now covered the tools you'll need for 90% of your layout needs, but there are more. A few are worth a glance just so you know what to look for should the situation come up. These widgets are designed for very particular layout situations that, while common, aren't everyday but need specialized tools to make happen.

Stack widget

This is for when you want to layer widgets so that they overlap one another. You want to stack them in the Z-direction. With Stack, you'll list some number of widgets, and they'll be displayed in that order one on top of another. The last one will occult (hide) the previous one if they overlap which will occult the one before that which will overlap the one before that and so on. I was really torn about where to cover the stack widget. On one hand, it involves laying out a screen which fits much better in this chapter.

GridView widget

Here's another thing borrowed from HTML and the Web. GridView is for displaying a list of widgets when you want them to appear in rows and columns but don't particularly care which rows and which columns – you just want them to show up in a grid. To use a GridView, you'll set its children property to the list of widgets you want to display and it will create the grid populating across and then wrapping to the next row, resizing its space available until it just fits. And here's the greatest part, it automatically scrolls! GridView has two constructors, GridView.extent() and GridView.count()

GridView.extent()

Extent refers to the maximum width of the child. GridView will only let its kids grow to that size. If they try to get bigger, it puts another element on that row and shrinks them all until they just fit across the entire width. Take a look at this example:

```
Widget build(BuildContext context) {
  return GridView.extent(
    maxCrossAxisExtent: 300.0,
    children:
      people.map<Widget>((dynamic person) =>
                          PersonCard(person)).toList(),
  );
}
```

Notice in Figures 4-12 and 4-13 how the containers resize to something less than 300. GridView decides that it can fit two across in portrait orientation. But when rotated, those two would have resized to something bigger than 300 so it puts three on each row.

Figure 4-12. GridView.extent() in portrait

Figure 4-13. The same GridView.extent() in landscape mode

The Table widget

The GridView is great when displaying widgets in rows and columns that wrap. The wrapping part means that you really don't care what children widgets end up in which row and column. Rows and Columns are best when you do care in which row and column the children exist. They're rigid when you want them to be. Unfortunately, the columns can't talk to each other so they will often be misaligned (Figures 4-13 and 4-14)

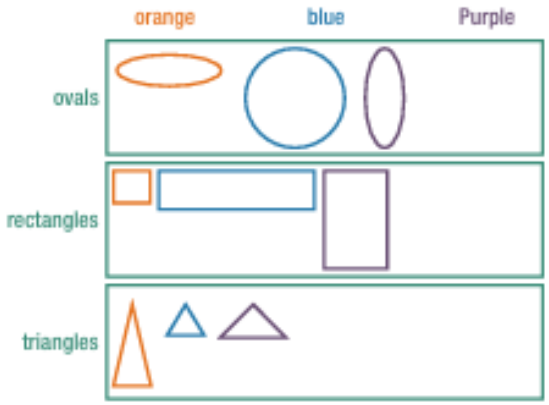

Figure 4-13. Rows work but the columns are misaligned

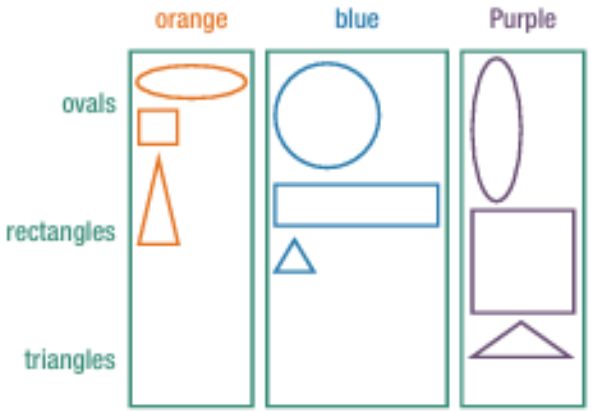

Figure 4-14. Columns work but the rows are misaligned

The Table widget fixes that problem. It is rigid like nested Rows and Columns, but each row and column is aware of the others and lines up nicely like GridView (Figure 4-15).

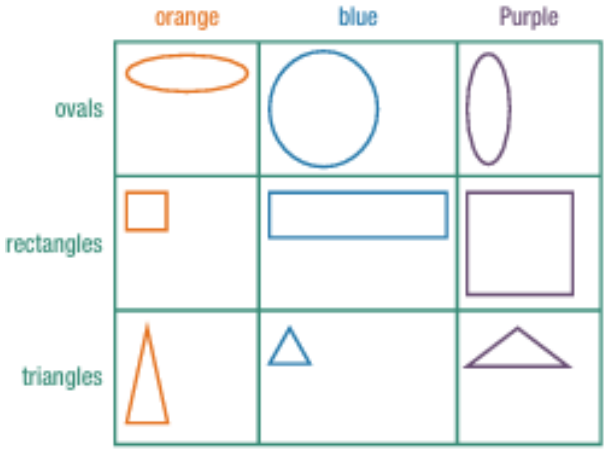

Figure 4-15. A Table aligns the rows and columns

Every Table widget will have children, a List of TableRow widgets. And each TableRow widget will have children, a List of widgets:

```
  return Table(
    children: <TableRow>[
          TableRow(children: <Widget>[
            Text('Salesperson', style: bold,),
            Text('January', style: bold,),

      Text('February', style: bold,),
      Text('March', style: bold,),
    ]
    ),
    TableRow(children: <Widget>[
      Text('Dwight'),
      Text('3742'),
      Text('5573'),
      Text('4323'),
    ],),
    TableRow(children: <Widget>[
      Text('Phyllis'),
      Text('3823'),
      Text('4500'),
      Text('3277'),
    ],
    ),
  ],
);
```

Chapter 5

Navigation and Routing

5.1 Introduction

All apps have the concept of moving from one screen to another. The user clicks the cart button, and we go to the card screen. The user clicks “continue shopping” button, and we get to browse for more products to buy. Some app developers call it routing. Others call it navigation. Whatever you want to call it, this is one area that Flutter makes really easy because there are only four ways of navigating:

• Stacks – Each widget is full screen. The user taps a button to go through a predefined workflow. History is maintained, and they can travel back one level by hitting a back button.

• Drawers – Most of the screen shows a widget, but on the left edge, a drawer is peeking out at the user. When they press it or swipe it right, it slides out revealing a menu of choices. Pressing one changes the widget in the main part of the screen.

• Tabs – Some room is reserved for a set of tabs at the top or the bottom of the screen. When you press on a tab, we show the widget that corresponds to that tab.

• Dialogs – While these aren’t technically part of navigation, they are a way to see another widget, so we’ll allow it. Dialogs are modal (aka pop-up windows) that stay until the user dismisses them.

Each of these methods depends on your app having a Material Widget as its ancestor. Let’s drill into them starting with stack navigation

5.2 Stack navigation

If you’re an experienced developer, you’re familiar with queues and stacks. If not, let me explain briefly. Let’s say you work in a kitchen. As plates are cleaned, they’re stacked, right? Each plate is put on the top of the stack. This is called pushing onto the stack. When it is time to serve some food, you naturally take the last plate added, the one on top of the stack. This is called popping off the top of the stack. Flutter’s

navigation works with stacks. When you want to send the user to a new scene, you will push() a widget on the top of the stack and the user sees that widget. Each time you push(), you're making the stack of scenes taller and taller. When you are ready for them to go back to where they were before, you'll pop() the last scene off the top of the stack, and what is revealed? The previous scene. With Flutter's stack, you'll typically predefine the scenes (aka routes) and give each a name. This must be done at the MaterialApp level like **so**

```
Widget build(BuildContext context) {
  return MaterialApp(
    title: 'Shopping App',
    initialRoute: '/',
    routes: {
      '/': (BuildContext ctx) => LandingScene(),
      '/browse': (BuildContext ctx) => Browse(),
      '/product': (BuildContext ctx) => ViewProduct(),

     '/checkout: (BuildContext ctx) => Checkout(),
   },
 );
}
```

Note that with routing, we no longer use the home property. Instead, use the intialRoute property.

Tip if your initialroute is "/", you can omit it altogether and Flutter assumes it is "/".

5.3 Navigating forward and back

To navigate the user to a scene manually, you'll Navigator. pushNamed(context, route) and Navigator.pop(context). To push a user to another route:

```
RaisedButton(
  child: const Text('Check out'),
  onPressed: () => Navigator.pushNamed(context, '/checkout),
),
```

Once they're finished and want to go back:

```
RaisedButton(
  child: const Text('Go back'),
  onPressed: () => Navigator.pop(context),
),
```

But wait, there's more! Notice that if you have a Scaffold, a back arrow is automatically added to the appbar (Figure 5-1). When tapped, it works to go back. And if your user is on Android, the ubiquitous Android back button works also (Figure 5-2).

Figure 5-1. The back arrow in the appbar

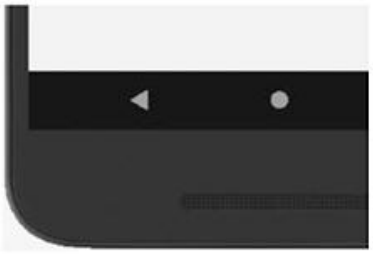

Figure 5-2. The Android back button works with stacks

Tip there is another flavor of routing that doesn't use a predefined routing table in your Materialapp. instead, you generate the route on the fly:

```
Navigator.push<void>(context, MaterialPageRoute
<void>(builder: (BuildContext context) =>
SecondRoute());
```

Drawer navigation

Drawers are great when we have a lot of navigation choices – too many choices to fit in a tab. In a lot of responsive web sites, you'll see a menu across the top of the page with links to other pages on the site. Then when the site is viewed on a small device or even a narrow browser, that menu is replaced by a hamburger menu that, when clicked, will drop down a menu filled with the same choices. Basically, this is the site responding to limited screen real estate, providing menu choices that are hidden until the user asks for them. Since most phones already have limited screen real estate, you may opt to put your menu choices in a drawer that doesn't gobble up that precious screen real estate until the user is ready to see them (Figure 5-3). When he or she is ready, they'll hit the now-familiar hamburger menu (that icon with three lines) and the choices slide out from the left (Figure 5-4). When the user chooses one, we'll Navigator.push() them to a new route.

Figure 5-3. *A scene with the drawer closed*

Figure 5-4. A scene with the drawer open

The Drawer widget

You'll need a Drawer widget, a built-in Flutter widget that has the ability to slide out, slide in, and contain menu choices. When you use a drawer, you always include it in a Scaffold's drawer property, like this:

```
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
```

```
      title: const Text('Drawer Navigation'),
    ),
    body: const Text('DrawerNavigation'),
    drawer: Drawer(child: ListView(
      children: <Widget>[
        Text('Option 1'),
        Text('Option 2'),
        Text('Option 3'),
      ],
    ),),
  );
}
```

Notice that when you have a drawer in your Scaffold, its hamburger icon replaces the appbar's back button. You can't see both buttons simultaneously unless you create your own buttons manually. So while drawer navigation and stack navigation can work together, it can be kind of awkward if you're not careful. One example of them working really well together is to have a Drawer at the topmost level, and then use stack navigation at all levels below that.

Tip

do you want a consistent drawer to be available across your entire app? if so, we generally put a Scaffold on every scene and include the drawer in it. therefore, it is best to put your drawer in its own widget and include it:

```
return Scaffold(
    appBar: AppBar(
      title: const Text('Drawer Navigation'),
    ),
  body: const Text('DrawerNavigation'),
  drawer: MyDrawer(),
);
```

Filling the drawer

Adding the drawer is easy. The trick is getting entries into the drawer and then making them navigate to another widget. Note that Drawer has a child property that accepts a single widget. To get multiple children in your drawer, you will use a widget that supports them such as Column (doesn't scroll) or List View (scrolls). Whichever you choose, you'll want to put something that is tap able

because to navigate, you're going to call Navigator.push() or Navigator. pushNamed() just like you did with stack navigation.

Tip there's a cool widget called a drawerheader that is built to take up a large area at the top of the drawer. it is great for putting your logo or other branding information to sort of remind the user what app they are in. it is cosmetic only but it really does look cool.

Tab Navigation

As you would imagine, a tab system matches N tabs with N widgets. When the user presses tab 1, they see widget 1 and so forth (Figure 5-5). The matching is done with a TabBar widget, a TabBarView widget, and a TabBarController.

Figure 5-5. A tabbar at the top and at the bottom

TabController

The TabController is the least obvious part. Just know that you have to have one

or you get the error in Figure 5-6

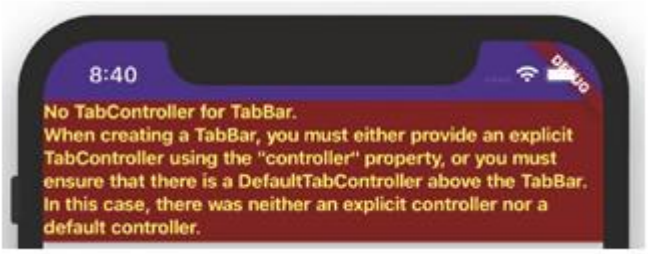

Figure 5-6. When you forget a TabController

The easiest way to create one is to wrap everything in a DefaultTabController() with a length property. Problem solved. This part is pretty simple – so simple you may wonder why Flutter doesn't create one implicitly for you. If you were thinking that, you wouldn't be wrong:

```
Widget build(BuildContext context) {
  return DefaultTabController(
    length: 3,
    child: Scaffold(
    ...
  );
}
```

The Dialog widget

Our last navigation category is arguably not a navigation category at all – dialogs. In one sense, you're showing another widget so ... navigation? But in another sense, you're basically showing a pop-up so ... not navigation. ¯_(ツ)_/¯

Either way, dialogs are a common thing and we should cover them. Since they don't fit well anywhere else in the book, let's pretend for the moment that they are a navigation topic. Hey, work with me here.

showDialog( ) and AlertDialog

showDialog() is a built-in Flutter method. You must supply a context and a builder method that returns a Widget, usually either SimpleDialog or AlertDialog. The AlertDialog has an actions parameter – a List of (typically) FlatButtons that let the user dismiss the dialog (Figure 5-7)

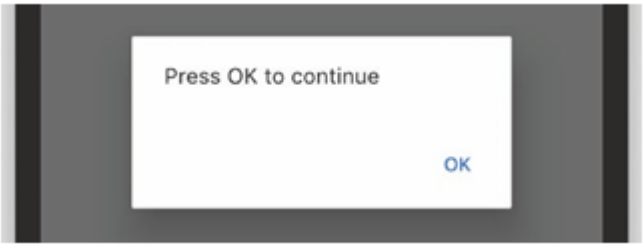

Figure 5-7. A simple AlertDialog

```
RaisedButton(
  child: const Text('I am a button. Press me'),
  onPressed: () => showDialog<void>(
    context: context,
    builder: (BuildContext context) {
      return AlertDialog(
        content: const Text('Press OK to continue'),
        actions: <Widget>[
          FlatButton(
              child: const Text('OK'),
              onPressed: () => Navigator.pop(context)),
        ],
      );


      },
    ),
  ),
```

This looks more complex than it needs to be. And this is the simplest form! It gets more complex if you want to give the user choices.

Responses with a Dialog

showDialog() returns a Future which means that you can have it return a value to its caller. Let's pretend you want the user to respond with yes or no (Figure 5-8)

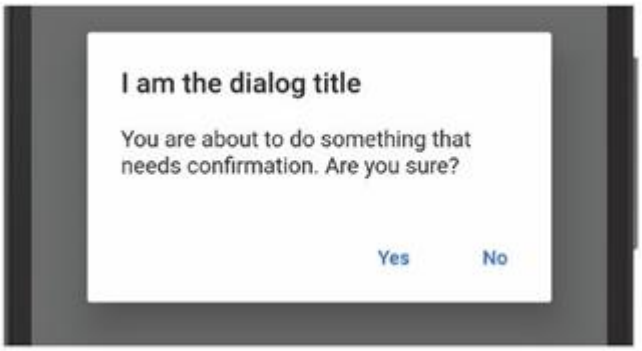

Figure 5-8. AlertDialog that returns a value

Navigation methods can be combined

While you can stack navigate to a widget with a drawer and from there to a widget with a tab, you should be careful. The methods are not incompatible, but, boy, they can get complex when mixed! For example, if you stack navigate via push() to a widget with a drawer, the back button in the appbar is no longer available. Android has a soft back button at the bottom, but iOS does not. So the user is now stuck with no way to return. Another example, a TabBarView has widgets, but these are hosted so to speak so they should have no Scaffold. If you tried to navigate to that same widget using either of the other two methods, you have no way to get back ... no drawer to show and no back button to tap. Again, the user is stuck. We recommend sticking to just two different types and keeping the levels consistent. For example, it is pretty common to have a tabbed navigation experience for the user, and within each tab, you'll work with stack navigation. But get much more complex than that and you may get your hands full.

References

1- **Payne, R. (2019). Beginning app development with flutter: Create cross-platform mobile apps. Apress.**